```r
pacman::p_load(
  dplyr, tidyr, readr, lme4, lmerTest,
  emmeans, here, ggplot2, clubSandwich
)
emm_options(lmerTest.limit = 5000)
emm_options(pbkrtest.limit = 5000)

PARAM_COLS <- c(
  paste0("p8_", 1:3),
  paste0("q_", 1:6),
  "N9", "v9"
)

MAIN_PARAMS <- c(
  p8_1 = "Integrated",
  p8_2 = "Heuristic",
  p8_3 = "Random",
  N9 = "N",
  v9 = "v"
)

Q_LABELS <- c(
  q_1 = "BO",
  q_2 = "HO",
  q_3 = "FO",
  q_4 = "JO",
  q_5 = "LS",
  q_6 = "50%"
)

raw <- read_csv(
  here::here("data", "Exp_2.csv"),
  show_col_types = FALSE
)

mf <- raw |>
  filter(block_num == 1) |>
  transmute(
    subject_id = as.character(subject_id),
    format = factor(format, levels = c("frequency", "probability")),
    time_pressure = factor(block_type, levels = c("Self-paced", "Time-limited")),
    judgment = inputvalue / 100,
    bayes = correctAnswer / 100,
    BO = br,
    HO = hr,
    FO = 1 - far,
    JO = br * hr,
    LS = hr - far
  ) |>
  mutate(
    abs_error = abs(judgment - bayes),
    match_Bayes = round(judgment, 2) == round(bayes, 2),
    match_BO = round(judgment, 2) == round(BO, 2),
```

```
    match_HO = round(judgment, 2) == round(HO, 2),
    match_FO = round(judgment, 2) == round(FO, 2),
    match_JO = round(judgment, 2) == round(JO, 2),
    match_LS = round(judgment, 2) == round(LS, 2),
    match_50 = round(judgment, 2) == 0.5,
    exact_any =
      match_Bayes |
      match_BO |
      match_HO |
      match_FO |
      match_JO |
      match_LS |
      match_50
  )

m_judgment <- lmer(
  judgment ~ format * time_pressure + (1 | subject_id),
  data = mf
)

judgment_anova <- anova(m_judgment, type = 3)
judgment_emm <- emmeans(m_judgment, ~ time_pressure | format)
judgment_contrasts <- contrast(
  judgment_emm,
  method = list("TL - SP" = c(-1, 1))
)

m_error <- lmer(
  abs_error ~ format * time_pressure + (1 | subject_id),
  data = mf
)

error_anova <- anova(m_error, type = 3)
error_emm <- emmeans(m_error, ~ time_pressure | format)
error_contrasts <- contrast(
  error_emm,
  method = list("TL - SP" = c(-1, 1))
)

individual_slopes <- mf |>
  group_by(subject_id, format, time_pressure) |>
  group_modify(
    ~ {
      fit <- lm(judgment ~ bayes, data = .x)
      tibble(
        intercept = coef(fit)[1],
        slope = coef(fit)[2]
      )
    }
  ) |>
  ungroup()

individual_slope_summary <- individual_slopes |>
```

```
  group_by(format, time_pressure) |>
  summarise(
    n = n(),
    mean = mean(slope),
    sd = sd(slope),
    median = median(slope),
    .groups = "drop"
  )

m_conservatism <- lmer(
  judgment ~ bayes * format * time_pressure +
    (1 + bayes | subject_id),
  data = mf
)

conservatism_anova <- anova(m_conservatism, type = 3)
conservatism_slopes <- emtrends(
  m_conservatism,
  ~ format * time_pressure,
  var = "bayes"
)
conservatism_contrasts <- contrast(
  conservatism_slopes,
  method = list("TL - SP" = c(-1, 1)),
  by = "format"
)

mf_nonexact <- mf |>
  filter(!exact_any)

m_nonexact <- lmer(
  judgment ~ bayes * format * time_pressure +
    (1 + bayes | subject_id),
  data = mf_nonexact
)

nonexact_anova <- anova(m_nonexact, type = 3)
nonexact_slopes <- emtrends(
  m_nonexact,
  ~ format * time_pressure,
  var = "bayes"
)
nonexact_contrasts <- contrast(
  nonexact_slopes,
  method = list("TL - SP" = c(-1, 1)),
  by = "format"
)

match_cols <- c(
  "match_Bayes", "match_BO", "match_HO",
  "match_FO", "match_JO", "match_LS", "match_50"
)
```

```r
match_props <- mf |>
  mutate(
    n_matches = rowSums(across(all_of(match_cols))),
    across(
      all_of(match_cols),
      ~ if_else(n_matches > 0, as.numeric(.x) / n_matches, 0)
    )
  ) |>
  dplyr::select(subject_id, format, time_pressure, all_of(match_cols)) |>
  pivot_longer(
    all_of(match_cols),
    names_to = "match_type",
    values_to = "match"
  ) |>
  mutate(
    match_type = dplyr::recode(
      match_type,
      match_Bayes = "Bayes",
      match_BO = "BO",
      match_HO = "HO",
      match_FO = "FO",
      match_JO = "JO",
      match_LS = "LS",
      match_50 = "50%"
    )
  ) |>
  group_by(subject_id, format, time_pressure, match_type) |>
  summarise(match_prop = mean(match), .groups = "drop")

match_types <- c("Bayes", "BO", "HO", "FO", "JO", "LS", "50%")

match_models <- lapply(match_types, function(mt) {
  lm(
    match_prop ~ format * time_pressure,
    data = match_props |> filter(match_type == mt)
  )
})

names(match_models) <- match_types

heuristic_props <- match_props |>
  filter(match_type != "Bayes") |>
  group_by(subject_id, format, time_pressure) |>
  summarise(
    match_prop = sum(match_prop),
    .groups = "drop"
  )

m_heuristic <- lm(
  match_prop ~ format * time_pressure,
  data = heuristic_props
)
```

```r
heuristic_anova <- anova(m_heuristic)

heuristic_emm <- emmeans(
  m_heuristic,
  ~ time_pressure | format
)

heuristic_contrasts <- contrast(
  heuristic_emm,
  method = list("TL - SP" = c(-1, 1))
)

posterior_path <- here::here("data", "Exp2_ML_Posterior_Draws_r05.rds")
posterior_draws <- readRDS(posterior_path)

first_blocks <- raw |>
  dplyr::filter(block_num == 1) |>
  dplyr::transmute(
    subject = as.character(subject_id),
    time_pressure = as.character(block_type),
    subject_block = paste(subject_id, block_type, sep = "__"),
    format = as.character(format)
  ) |>
  dplyr::distinct()

first_draws <- posterior_draws |>
  dplyr::semi_join(first_blocks, by = "subject_block") |>
  dplyr::select(subject_block, dplyr::all_of(PARAM_COLS)) |>
  dplyr::left_join(first_blocks, by = "subject_block") |>
  dplyr::mutate(
    format = factor(format, levels = c("frequency", "probability")),
    time_pressure = factor(time_pressure, levels = c("Self-paced", "Time-limited"))
  )

posterior_medians <- first_draws |>
  dplyr::group_by(subject_block, subject, format, time_pressure) |>
  dplyr::summarise(
    dplyr::across(dplyr::all_of(PARAM_COLS), ~ median(.x, na.rm = TRUE)),
    .groups = "drop"
  )

parameter_long <- posterior_medians |>
  tidyr::pivot_longer(dplyr::all_of(PARAM_COLS),
                      names_to = "parameter", values_to = "value")

between_test <- function(data, group_col = "parameter") {
  data |>
    dplyr::group_by(dplyr::across(dplyr::all_of(c(group_col, "format")))) |>
    dplyr::group_modify(~ {
      sp <- .x$value[.x$time_pressure == "Self-paced"]
      tl <- .x$value[.x$time_pressure == "Time-limited"]
      tt <- stats::t.test(tl, sp)
      tibble::tibble(
```

```
        n_SP = length(sp), n_TL = length(tl),
        mean_SP = mean(sp), mean_TL = mean(tl),
        difference_TL_minus_SP = mean(tl) - mean(sp),
        t = unname(tt$statistic), df = unname(tt$parameter),
        p = tt$p.value, CI_low = tt$conf.int[1], CI_high = tt$conf.int[2]
      )
    }) |>
    dplyr::ungroup()
}

describe_params <- function(data, group_col = "parameter") {
  data |>
    dplyr::group_by(dplyr::across(dplyr::all_of(c(group_col, "format", "time_pressure")))) |>
    dplyr::summarise(n = dplyr::n(), mean = mean(value), sd = sd(value),
                     median = median(value), .groups = "drop")
}

main_long <- parameter_long |>
  dplyr::filter(parameter %in% names(MAIN_PARAMS)) |>
  dplyr::mutate(parameter = dplyr::recode(parameter, !!!MAIN_PARAMS))
main_parameter_descriptives <- describe_params(main_long)
main_parameter_tests <- between_test(main_long)

q_long <- parameter_long |>
  dplyr::filter(parameter %in% names(Q_LABELS)) |>
  dplyr::mutate(heuristic = dplyr::recode(parameter, !!!Q_LABELS))
q_descriptives <- describe_params(q_long, "heuristic")
q_tests <- between_test(q_long, "heuristic")

options(width = 200, digits = 8)
cat("\n=== First-block sample ===\n")
```

```
##
## === First-block sample ===
```

```
cat("Behavioural participants:", dplyr::n_distinct(mf$subject_id), "; trials:", nrow(mf), "\n")
```

```
## Behavioural participants: 83 ; trials: 4281
```

```
print(mf |> dplyr::distinct(subject_id, format, time_pressure) |> dplyr::count(format, time_pressure, na
```

```
## # A tibble: 4 x 3
##   format      time_pressure n_participants
##   <fct>       <fct>                  <int>
## 1 frequency   Self-paced                15
## 2 frequency   Time-limited              19
## 3 probability Self-paced                24
## 4 probability Time-limited              25
```

```
cat("ABC participants:", dplyr::n_distinct(posterior_medians$subject), "\n")
```

```
## ABC participants: 83
```

```
print(posterior_medians |> dplyr::count(format, time_pressure, name = "n_participants"), n = Inf)
```

```
## # A tibble: 4 x 3
##   format      time_pressure n_participants
##   <fct>       <fct>                  <int>
## 1 frequency   Self-paced                15
## 2 frequency   Time-limited              19
## 3 probability Self-paced                24
## 4 probability Time-limited              25
```

```
cat("\n=== Judgment accuracy ===\n")
```

```
##
## === Judgment accuracy ===
```

```
print(error_anova)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##                         Sum Sq  Mean Sq NumDF   DenDF  F value     Pr(>F)
## format                0.023867 0.023867     1 79.1188  0.50745   0.478340
## time_pressure         0.838338 0.838338     1 79.1188 17.82405 6.4181e-05 ***
## format:time_pressure 0.160810 0.160810     1 79.1188  3.41901   0.068185 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
print(summary(error_emm, infer = TRUE))
```

```
## format = frequency:
##  time_pressure emmean     SE   df lower.CL upper.CL t.ratio p.value
##  Self-paced     0.136 0.0317 78.5   0.0724    0.199   4.271  0.0001
##  Time-limited   0.303 0.0283 79.3   0.2467    0.359  10.718  <.0001
##
## format = probability:
##  time_pressure emmean     SE   df lower.CL upper.CL t.ratio p.value
##  Self-paced     0.206 0.0251 78.5   0.1562    0.256   8.219  <.0001
##  Time-limited   0.272 0.0247 79.6   0.2225    0.321  11.013  <.0001
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
print(summary(error_contrasts, infer = TRUE))
```

```
## format = frequency:
##  contrast estimate     SE   df lower.CL upper.CL t.ratio p.value
##  TL - SP    0.1675 0.0425 78.8  0.08287    0.252   3.941  0.0002
##
## format = probability:
##  contrast estimate     SE   df lower.CL upper.CL t.ratio p.value
```

```
##  TL - SP    0.0655 0.0352 79.0 -0.00457    0.135   1.861  0.0665
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
cat("\n=== Posterior sensitivity ===\n")
```

```

##
## === Posterior sensitivity ===
```

```
print(conservatism_anova)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##                              Sum Sq Mean Sq NumDF   DenDF   F value     Pr(>F)
## bayes                       5.06778 5.06778     1 79.0322 151.79393 < 2.22e-16 ***
## format                      0.05618 0.05618     1 79.0664   1.68285 0.19831804
## time_pressure               0.06651 0.06651     1 79.0664   1.99227 0.16202665
## bayes:format                0.02658 0.02658     1 79.0322   0.79607 0.37497847
## bayes:time_pressure         0.39053 0.39053     1 79.0322  11.69748 0.00099291 ***
## format:time_pressure        0.00177 0.00177     1 79.0664   0.05291 0.81866974
## bayes:format:time_pressure 0.06275 0.06275     1 79.0322   1.87939 0.17428370
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
print(summary(conservatism_slopes, infer = TRUE))
```

```
##  format      time_pressure bayes.trend     SE   df lower.CL upper.CL t.ratio p.value
##  frequency   Self-paced          0.703 0.0899 78.7    0.524    0.882   7.821  <.0001
##  probability Self-paced          0.526 0.0710 78.7    0.385    0.667   7.406  <.0001
##  frequency   Time-limited        0.329 0.0800 79.2    0.170    0.488   4.111  0.0001
##  probability Time-limited        0.366 0.0697 79.3    0.227    0.505   5.249  <.0001
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
print(summary(conservatism_contrasts, infer = TRUE))
```

```
## format = frequency:
##  contrast estimate     SE   df lower.CL upper.CL t.ratio p.value
##  TL - SP    -0.374 0.1203 78.9   -0.613  -0.1346  -3.110  0.0026
##
## format = probability:
##  contrast estimate     SE   df lower.CL upper.CL t.ratio p.value
##  TL - SP    -0.160 0.0996 79.0   -0.358   0.0382  -1.607  0.1120
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
cat("\n=== Non-exact responses ===\n")
```

```
##
## === Non-exact responses ===
```

```
cat("Participants:", dplyr::n_distinct(mf_nonexact$subject_id), "; trials:", nrow(mf_nonexact), "\n")
```

```
## Participants: 72 ; trials: 2215
```

```
print(nonexact_anova)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##                             Sum Sq Mean Sq NumDF   DenDF  F value     Pr(>F)
## bayes                      4.06687 4.06687     1 62.7072 96.16243 2.8061e-14 ***
## format                     0.01432 0.01432     1 61.1548  0.33852   0.562825
## time_pressure              0.25479 0.25479     1 61.1548  6.02467   0.016968 *
## bayes:format               0.00074 0.00074     1 62.7072  0.01753   0.895083
## bayes:time_pressure        0.86705 0.86705     1 62.7072 20.50166 2.7246e-05 ***
## format:time_pressure       0.06194 0.06194     1 61.1548  1.46452   0.230869
## bayes:format:time_pressure 0.00049 0.00049     1 62.7072  0.01161   0.914521
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
print(summary(nonexact_slopes, infer = TRUE))
```

```
##  format      time_pressure bayes.trend     SE   df lower.CL upper.CL t.ratio p.value
##  frequency   Self-paced          0.534 0.0885 67.8   0.3571    0.711   6.029  <.0001
##  probability Self-paced          0.516 0.0679 68.2   0.3808    0.652   7.606  <.0001
##  frequency   Time-limited        0.194 0.0678 58.2   0.0585    0.330   2.864  0.0058
##  probability Time-limited        0.192 0.0669 64.1   0.0588    0.326   2.876  0.0055
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
print(summary(nonexact_contrasts, infer = TRUE))
```

```
## format = frequency:
##  contrast estimate     SE   df lower.CL upper.CL t.ratio p.value
##  TL - SP    -0.340 0.1115 64.0   -0.562   -0.117  -3.045  0.0034
##
## format = probability:
##  contrast estimate     SE   df lower.CL upper.CL t.ratio p.value
##  TL - SP    -0.324 0.0953 66.1   -0.514   -0.134  -3.397  0.0012
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
print(summary(nonexact_slopes, null = 0, infer = c(TRUE, TRUE)))
```

```
##  format      time_pressure bayes.trend     SE   df lower.CL upper.CL t.ratio p.value
##  frequency   Self-paced          0.534 0.0885 67.8   0.3571    0.711   6.029  <.0001
##  probability Self-paced          0.516 0.0679 68.2   0.3808    0.652   7.606  <.0001
```

```
##  frequency   Time-limited        0.194 0.0678 58.2   0.0585    0.330   2.864  0.0058
##  probability Time-limited        0.192 0.0669 64.1   0.0588    0.326   2.876  0.0055
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
print(summary(nonexact_slopes, null = 1, infer = c(TRUE, TRUE)))
```

```
##  format      time_pressure bayes.trend     SE   df lower.CL upper.CL null t.ratio p.value
##  frequency   Self-paced          0.534 0.0885 67.8   0.3571    0.711    1  -5.265  <.0001
##  probability Self-paced          0.516 0.0679 68.2   0.3808    0.652    1  -7.128  <.0001
##  frequency   Time-limited        0.194 0.0678 58.2   0.0585    0.330    1 -11.879  <.0001
##  probability Time-limited        0.192 0.0669 64.1   0.0588    0.326    1 -12.070  <.0001
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
cat("\n=== Exact Bayesian matching ===\n")
```

```
##
## === Exact Bayesian matching ===
```

```
print(anova(match_models[["Bayes"]]))
```

```
## Analysis of Variance Table
##
## Response: match_prop
##                       Df  Sum Sq  Mean Sq  F value    Pr(>F)
## format                 1 0.07746 0.077457  1.65956   0.20142
## time_pressure          1 1.01492 1.014919 21.74524 1.247e-05 ***
## format:time_pressure   1 0.10090 0.100897  2.16178   0.14545
## Residuals             79 3.68718 0.046673
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
bayes_emm <- emmeans(match_models[["Bayes"]], ~ time_pressure | format)
print(summary(bayes_emm, infer = TRUE))
```

```
## format = frequency:
##  time_pressure emmean     SE df lower.CL upper.CL t.ratio p.value
##  Self-paced    0.3309 0.0558 79   0.2198    0.442   5.931  <.0001
##  Time-limited  0.0245 0.0496 79  -0.0741    0.123   0.495  0.6222
##
## format = probability:
##  time_pressure emmean     SE df lower.CL upper.CL t.ratio p.value
##  Self-paced    0.1812 0.0441 79   0.0934    0.269   4.109  0.0001
##  Time-limited  0.0172 0.0432 79  -0.0688    0.103   0.399  0.6908
##
## Confidence level used: 0.95
```

```r
print(summary(contrast(bayes_emm, method = list("TL - SP" = c(-1, 1))), infer = TRUE))
```

```
## format = frequency:
##  contrast estimate     SE df lower.CL upper.CL t.ratio p.value
##  TL - SP    -0.306 0.0746 79   -0.455  -0.1578  -4.105  0.0001
##
## format = probability:
##  contrast estimate     SE df lower.CL upper.CL t.ratio p.value
##  TL - SP    -0.164 0.0617 79   -0.287  -0.0411  -2.656  0.0096
##
## Confidence level used: 0.95
```

```r
cat("\n=== Overall heuristic matching ===\n")
```

```
##
## === Overall heuristic matching ===
```

```r
print(heuristic_anova)
```

```
## Analysis of Variance Table
##
## Response: match_prop
##                      Df  Sum Sq  Mean Sq F value   Pr(>F)
## format                1 0.43386 0.433861 4.04122 0.047815 *
## time_pressure         1 0.17630 0.176305 1.64220 0.203772
## format:time_pressure  1 0.00200 0.001997 0.01860 0.891854
## Residuals            79 8.48135 0.107359
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
print(summary(heuristic_emm, infer = TRUE))
```

```
## format = frequency:
##  time_pressure emmean     SE df lower.CL upper.CL t.ratio p.value
##  Self-paced     0.226 0.0846 79   0.0575    0.394   2.671  0.0092
##  Time-limited   0.306 0.0752 79   0.1569    0.456   4.077  0.0001
##
## format = probability:
##  time_pressure emmean     SE df lower.CL upper.CL t.ratio p.value
##  Self-paced     0.367 0.0669 79   0.2335    0.500   5.482  <.0001
##  Time-limited   0.467 0.0655 79   0.3368    0.598   7.130  <.0001
##
## Confidence level used: 0.95
```

```r
print(summary(heuristic_contrasts, infer = TRUE))
```

```
## format = frequency:
##  contrast estimate     SE df lower.CL upper.CL t.ratio p.value
##  TL - SP    0.0806 0.1132 79  -0.1447    0.306   0.712  0.4787
##
```

```
## format = probability:
##  contrast estimate     SE df lower.CL upper.CL t.ratio p.value
##  TL - SP    0.1006 0.0936 79  -0.0858    0.287   1.074  0.2860
##
## Confidence level used: 0.95
```

```
cat("\n=== Main parameter estimates ===\n")
```

```
##
## === Main parameter estimates ===
```

```
print(main_parameter_descriptives, n = Inf, width = Inf)
```

```
## # A tibble: 20 x 7
##    parameter  format      time_pressure     n   mean       sd median
##    <chr>      <fct>       <fct>         <int>  <dbl>    <dbl>  <dbl>
##  1 Heuristic  frequency   Self-paced       15 0.399  0.158    0.402
##  2 Heuristic  frequency   Time-limited     19 0.533  0.125    0.565
##  3 Heuristic  probability Self-paced       24 0.497  0.232    0.485
##  4 Heuristic  probability Time-limited     25 0.571  0.111    0.555
##  5 Integrated frequency   Self-paced       15 0.576  0.159    0.576
##  6 Integrated frequency   Time-limited     19 0.440  0.125    0.406
##  7 Integrated probability Self-paced       24 0.478  0.232    0.487
##  8 Integrated probability Time-limited     25 0.404  0.110    0.417
##  9 N          frequency   Self-paced       15 3.93   1.10     4
## 10 N          frequency   Time-limited     19 3.05   0.705    3
## 11 N          probability Self-paced       24 4.04   1.20     4
## 12 N          probability Time-limited     25 3.16   0.8      3
## 13 Random     frequency   Self-paced       15 0.0259 0.000995 0.0263
## 14 Random     frequency   Time-limited     19 0.0270 0.00128  0.0270
## 15 Random     probability Self-paced       24 0.0259 0.00119  0.0262
## 16 Random     probability Time-limited     25 0.0265 0.00145  0.0263
## 17 v          frequency   Self-paced       15 0.497  0.275    0.395
## 18 v          frequency   Time-limited     19 0.536  0.162    0.522
## 19 v          probability Self-paced       24 0.477  0.196    0.486
## 20 v          probability Time-limited     25 0.593  0.168    0.556
```

```
print(main_parameter_tests, n = Inf, width = Inf)
```

```
## # A tibble: 10 x 12
##    parameter  format       n_SP  n_TL mean_SP mean_TL difference_TL_minus_SP      t    df       p
##    <chr>      <fct>       <int> <int>   <dbl>   <dbl>                 <dbl>  <dbl> <dbl>   <dbl>
##  1 Heuristic  frequency      15    19  0.399   0.533                 0.133     2.67   26.2 0.0129   0
##  2 Heuristic  probability    24    25  0.497   0.571                 0.0742    1.42   32.7 0.165   -0
##  3 Integrated frequency      15    19  0.576   0.440                -0.136    -2.71   26.2 0.0116  -0
##  4 Integrated probability    24    25  0.478   0.404                -0.0741   -1.42   32.5 0.166   -0
##  5 N          frequency      15    19  3.93    3.05                 -0.881    -2.69   22.7 0.0130  -1
##  6 N          probability    24    25  4.04    3.16                 -0.882    -3.02   39.9 0.00441 -1
##  7 Random     frequency      15    19  0.0259  0.0270                0.00109   2.79   32.0 0.00889  0
##  8 Random     probability    24    25  0.0259  0.0265                0.000596  1.58   45.9 0.122   -0
##  9 v          frequency      15    19  0.497   0.536                 0.0398    0.496  21.5 0.625   -0
## 10 v          probability    24    25  0.477   0.593                 0.117     2.24   45.3 0.0301   0
```

```r
cat("\n=== Heuristic anchor parameters ===\n")
```

```
##
## === Heuristic anchor parameters ===
```

```r
print(q_descriptives, n = Inf, width = Inf)
```

```
## # A tibble: 24 x 7
##    heuristic format      time_pressure     n   mean     sd median
##    <chr>     <fct>       <fct>         <int>  <dbl>  <dbl>  <dbl>
##  1 50%       frequency   Self-paced       15 0.127  0.0427 0.109
##  2 50%       frequency   Time-limited     19 0.169  0.0599 0.164
##  3 50%       probability Self-paced       24 0.128  0.0476 0.109
##  4 50%       probability Time-limited     25 0.181  0.0782 0.196
##  5 BO        frequency   Self-paced       15 0.145  0.0605 0.130
##  6 BO        frequency   Time-limited     19 0.161  0.0622 0.155
##  7 BO        probability Self-paced       24 0.138  0.0633 0.124
##  8 BO        probability Time-limited     25 0.154  0.0785 0.123
##  9 FO        frequency   Self-paced       15 0.118  0.0365 0.126
## 10 FO        frequency   Time-limited     19 0.0997 0.0358 0.0865
## 11 FO        probability Self-paced       24 0.123  0.0707 0.114
## 12 FO        probability Time-limited     25 0.101  0.0321 0.102
## 13 HO        frequency   Self-paced       15 0.165  0.0966 0.136
## 14 HO        frequency   Time-limited     19 0.117  0.0499 0.104
## 15 HO        probability Self-paced       24 0.172  0.115  0.123
## 16 HO        probability Time-limited     25 0.162  0.103  0.137
## 17 JO        frequency   Self-paced       15 0.151  0.0754 0.122
## 18 JO        frequency   Time-limited     19 0.188  0.0910 0.162
## 19 JO        probability Self-paced       24 0.151  0.0845 0.119
## 20 JO        probability Time-limited     25 0.161  0.0823 0.124
## 21 LS        frequency   Self-paced       15 0.119  0.0168 0.121
## 22 LS        frequency   Time-limited     19 0.126  0.0177 0.124
## 23 LS        probability Self-paced       24 0.126  0.0550 0.113
## 24 LS        probability Time-limited     25 0.113  0.0186 0.114
```

```r
print(q_tests, n = Inf, width = Inf)
```

```
## # A tibble: 12 x 12
##    heuristic format       n_SP  n_TL mean_SP mean_TL difference_TL_minus_SP      t    df       p    CI
##    <chr>     <fct>       <int> <int>   <dbl>   <dbl>                  <dbl>  <dbl> <dbl>   <dbl>    <
##  1 50%       frequency      15    19   0.127  0.169                 0.0418   2.37   31.7 0.0238   0.0
##  2 50%       probability    24    25   0.128  0.181                 0.0533   2.89   39.9 0.00619  0.0
##  3 BO        frequency      15    19   0.145  0.161                 0.0165   0.779  30.5 0.442   -0.0
##  4 BO        probability    24    25   0.138  0.154                 0.0158   0.778  45.7 0.441   -0.0
##  5 FO        frequency      15    19   0.118  0.0997               -0.0182  -1.46   29.9 0.156   -0.0
##  6 FO        probability    24    25   0.123  0.101                -0.0224  -1.42   31.8 0.167   -0.0
##  7 HO        frequency      15    19   0.165  0.117                -0.0478  -1.74   19.8 0.0973  -0.1
##  8 HO        probability    24    25   0.172  0.162                -0.0106  -0.338  46.0 0.737   -0.0
##  9 JO        frequency      15    19   0.151  0.188                 0.0364   1.28   31.9 0.211   -0.0
## 10 JO        probability    24    25   0.151  0.161                 0.0101   0.426  46.8 0.672   -0.0
## 11 LS        frequency      15    19   0.119  0.126                 0.00766  1.29   30.9 0.206   -0.0
## 12 LS        probability    24    25   0.126  0.113                -0.0126  -1.06   28.0 0.297   -0.0
```